Основы Git

Занятие 06 · День 3, после обеда

Версионируем манифесты — фундамент для CI/CD и GitOps

Зачем DevOps-инженеру Git

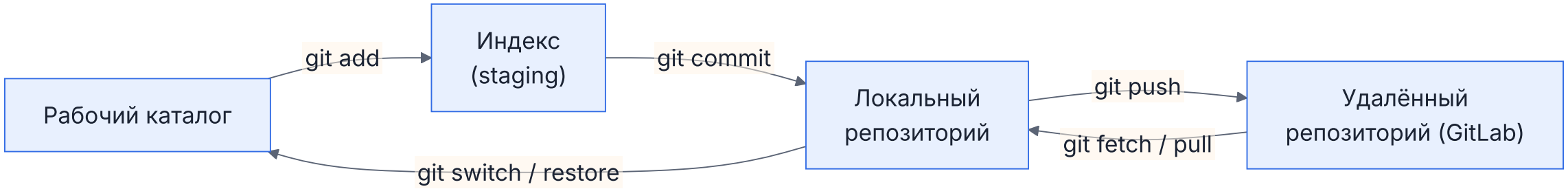
- **Всё как код**: приложение, Dockerfile, манифесты, Helm-чарты, `.gitlab-ci.yml`, inventory Kubespray
- История: **кто, когда и зачем** изменил
- Совместная работа: ветки, merge request, code review
- **Git — триггер CI/CD**: каждый `push` запускает пайплайн
- **GitOps**: состояние кластера = содержимое репозитория

Git — распределённая система

- У каждого — **полная копия** репозитория с историей
- Коммиты, ветки, слияния — **локально и мгновенно**, без сервера
- Сервер (GitLab, GitHub) — просто ещё одна копия, «точка встречи» команды
- Каждый коммит — снимок проекта, идентифицируется **SHA-хешем**

```
git --version
git config --global user.name  "Иван Петров"
git config --global user.email "ivan@example.com"
git config --global init.defaultBranch main
git config --global core.editor vim
git config --list
```

Три области Git




```
git status            # что изменено, что в индексе
git diff              # рабочий каталог ↔ индекс
git diff --staged     # индекс ↔ последний коммит
```

Первый репозиторий

```
mkdir ~/shop && cd ~/shop       # или каталог с манифестами со вчерашнего дня
git init
git status

git add k8s/deployment.yaml     # конкретный файл
git add .                       # всё
git commit -m "Добавлены манифесты shop-web"

git log --oneline --graph --decorate
git show HEAD
```

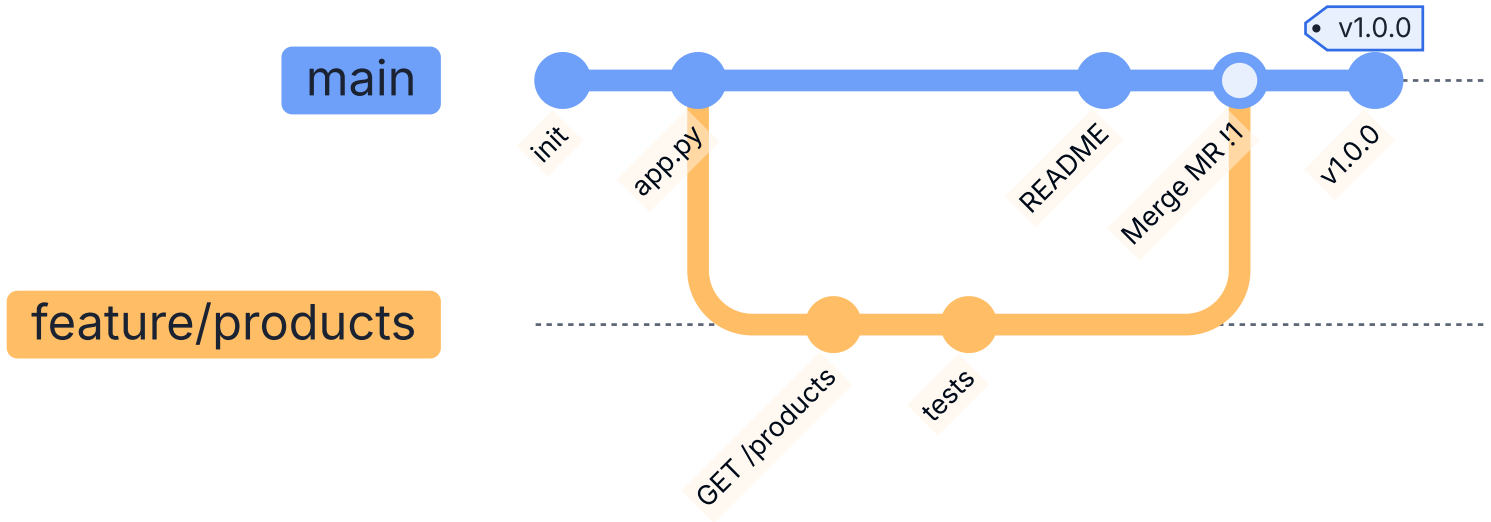
> Коммит — логически законченное изменение. «Добавил пробы», а не «изменения 2».

`.gitignore` и что НЕ коммитить

```
# секреты и доступы
*.kubeconfig
admin.conf
secret.yaml
.env
# мусор
__pycache__/
*.log
.vagrant/
.idea/
.vscode/
```

- Секрет **один раз** попал в историю — считайте его **скомпрометированным**: менять пароль/токен, а не «удалять коммит»
- `git rm --cached file` — перестать отслеживать файл, оставив на диске

Ветки




```
git branch                    # список
git switch -c feature/probes  # создать и перейти
git switch main               # вернуться
git branch -d feature/probes  # удалить слитую ветку
```

Ветка — просто **указатель** на коммит. Создание ветки ничего не копирует.

Слияние: merge и rebase

merge — сохраняет историю как была

```
git switch main
git merge feature/probes
```

Fast-forward, если `main` не ушла вперёд; иначе — **merge-коммит**.

rebase — «переносит» коммиты ветки поверх свежей `main`

```
git switch feature/probes
git rebase main
```

Линейная история. **Не делайте rebase опубликованных общих веток!**

В GitLab слияние делают через **Merge Request** — с review и пайплайном.

Конфликты

```
<<<<<<< HEAD
  replicas: 3
=======
  replicas: 5
>>>>>>> feature/scale
```

1. `git status` — какие файлы в конфликте
2. Открыть файл, оставить нужный вариант, **удалить маркеры**
3. `git add <файл>` → `git commit` (или `git rebase --continue`)
4. Передумали: `git merge --abort` / `git rebase --abort`

> Конфликт — не ошибка, а вопрос «какой вариант правильный?», который Git задаёт человеку.

Удалённые репозитории

```
git remote add origin http://gitlab.example/demo/shop.git
git remote -v

git push -u origin main          # отправить и связать ветку
git fetch                        # забрать изменения, НЕ трогая рабочий каталог
git pull                         # fetch + merge (или --rebase)
git clone <url>                  # получить копию
```

- `origin` — просто имя по соглашению
- Remote может быть даже **каталогом** на диске — покажем прямо сейчас, до установки GitLab

Теги и версии

```
git tag -a v1.0.0 -m "Первый релиз"
git push origin v1.0.0           # теги не уходят с обычным push!
git tag -l
git checkout v1.0.0              # посмотреть состояние на момент релиза
```

Semantic Versioning `MAJOR.MINOR.PATCH`:

- MAJOR — несовместимые изменения API
- MINOR — новая функциональность, совместимо
- PATCH — исправления

В CI тег часто запускает **релиз**: сборку версии и деплой в production.

Исправляем ошибки

Ситуация	Команда
Отменить правки в файле	`git restore file`
Убрать файл из индекса	`git restore --staged file`
Поправить последний коммит (ещё не отправлен)	`git commit --amend`
Откатить коммит в **общей** ветке	`git revert <sha>` — новый «обратный» коммит
Откатить локальные коммиты	`git reset --soft HEAD~1` / `--hard` (осторожно!)
«Я всё сломал»	`git reflog` — журнал всех положений HEAD

Правило: что уже в `push` у коллег — только `revert`, никаких `reset` + `push --force`.

Сообщения коммитов

```
feat(chart): добавить HPA для shop-api

Нагрузочный тест показал пики до 400 RPS.
HPA масштабирует от 2 до 6 реплик по CPU 60%.

Closes #42
```

Conventional Commits: `feat`, `fix`, `docs`, `ci`, `refactor`, `chore`...

- Первая строка ≤ 72 символов, повелительное наклонение
- Тело — **зачем**, а не «что» (что — видно в diff)
- `Closes #42` — GitLab сам закроет issue после слияния

Стратегии ветвления

Стратегия	Суть	Когда
GitHub flow	`main` + короткие feature-ветки + MR	большинство веб-проектов
GitLab flow	+ ветки/окружения или релизные ветки	несколько окружений, релизы
Trunk-based	очень короткие ветки, частые слияния, feature flags	зрелый CI, частые релизы
Git flow	`develop`, `release/*`, `hotfix/*`	редкие версионные релизы

На курсе: **`main` защищена**, работа в `feature/*`, слияние через **MR**, релиз — **тег** `vX.Y.Z`.

ДЕМО Демо: Git без сервера

```
# «сервер» — bare-репозиторий в каталоге
git init --bare ~/remote/shop.git

cd ~/shop
git init && git add . && git commit -m "feat: манифесты shop-web"
git remote add origin ~/remote/shop.git
git push -u origin main

# «коллега» клонирует и вносит правку
git clone ~/remote/shop.git /tmp/colleague && cd /tmp/colleague
git switch -c feature/replicas
sed -i 's/replicas: 3/replicas: 4/' k8s/deployment.yaml
git commit -am "feat: 4 реплики" && git push -u origin feature/replicas

# мы — забираем и сливаем
cd ~/shop && git fetch && git log --oneline --all --graph
git merge origin/feature/replicas && git push
kubectl apply -f k8s/ && kubectl -n shop get deploy
```

Итоги

- Три области: **рабочий каталог → индекс → репозиторий**, и удалённый репозиторий
- Ветки дешёвые: любая задача — в своей ветке
- `merge` vs `rebase`; общие ветки — только `revert`
- **Секреты не в Git**, `.gitignore` с первого коммита
- Теги + SemVer → релизы

Завтра этот же репозиторий переедет в **GitLab**, и `push` начнёт запускать пайплайны.

Практика 06

Git на практике

Задание — на платформе testlms.kz